ECE5755 / CS5754 Modern Computer Systems and Architecture, Fall 2025
Assignment 2: Implementing Tiling and Blocking

1. Introduction

This lab assignment will build off the work you completed in the previous lab assignment. In particular, you will be exploring the performance gains in your kernel when you implement **loop tiling/blocking** and **quantization** for matmul. You will also implement the **attention** kernel.

2. Materials

We have included the template for the attention kernel and one test case in **lab2.zip**. Since you need to build this up on your existing code, please copy the template to your own codebase. Make sure to make a copy of your original lab 1 code or use a version control tool like git so that you have a known good state to revert to. You will add two additional functions, `matmul_blocking and matmul_quant`, which implements tiling/blocking and quantized version of `matmul`, `respectively`.

3. Objective

3.1 Matmul with tiling/blocking

One objective of this lab is to implement a better matrix-multiplication algorithm that uses tiling/blocking, and **to analyze the performance gains of your algorithm using microarchitectural analysis techniques**. You are tasked with implementing the following function:

```
float **matmul_blocking(float **A, float **B, int A_rows, int A_cols,
int B_rows, int B_cols)
```

The function should be put in `kernel/matrix_ops.c`. Don't forget to put the function declaration in the header file (`kernel/matrix_ops.h`) as well so the lab will compile. You'll then want to swap out your original matrix multiplication implementation for this one in your tests to verify that it's actually working, i.e. call `matmul_blocking` instead of `matmul` inside `tests/test_matrix_ops.c`

You are expected to optimize `matmul_blocking` for performance (runtime). Try a few different implementations in terms of tiling/blocking sizes, patterns, and loop order and try to find the best implementation for performance. When submitting, only submit the

final/most optimized implementation. When testing for runtime, you can use the Linux `time` command; make sure to take the `real` time which is the wall-clock time. Moreover, as with profiling, make sure you are looping over the function you want to profile so that the runtime of the entire program is dominated by the function you are profiling/timing. Test with a variety of input sizes; **make sure you have some inputs that are large enough to fill up the L1 cache.**

For this task, we will not be grading your testing procedure, only the correctness of `matmul_blocking`. Functional correctness grading will be done with an internal grading program similar to the testing programs you were expected to submit for Lab 1. We will also be reading your code to grade for correctness in implementing tiling/blocking, so please make sure your code is readable (use comments to clarify your code).

Your profiling process must meet the following requirements:

- Use `toplev.py` from `pmu-tools` for Top-down analysis data
- Profile only the core the program is running on
- Runtime of the program-under-test should be dominated by the function-under-test (you can accomplish this by looping over your function many times)

The specific commands you use for profiling and testing and how you write your program-under-test is up to you as long as your process meets the above requirements. One option is to use the existing testing infrastructure under `tests/`.

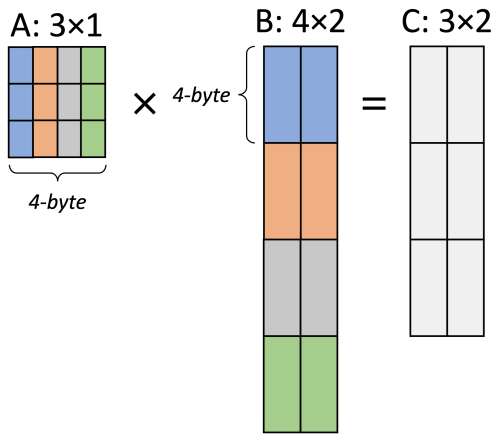
3.2 Matmul with quantization

In addition to tiling/blocking, we also ask you to implement a **quantized** matrix-multiplication algorithm C = Q_A*B, where the input matrix $\mathbf{Q_A}$ stores **8-bit unsigned integer** instead of the conventional 4-byte integer. You are tasked with implementing the following function:

```
float **matmul_quant(unsigned int **A, int **B, int A_rows, int
A_cols, int B_rows, int B_cols)
```

The function should be put in `kernel/matrix_ops.c`. Note the different data type used for each operand. The matrix A stores unsigned integers, i.e., every A[i][k] is a 4-byte unsigned integer. However, since A is quantized to 8-bit, every A[i][k] can store four 8-bit elements. This also indicates that **B_rows = 4 × A_cols** because B is unquantized. See the figure below for an example. Finally, the returned output matrix C

stores float numbers. You may choose to compute A*B in integer, and use type casting to convert the output to float.



An obvious advantage of quantization is to reduce the required storage space and corresponding memory access. Specifically, the input matrix A can be compressed by 4× when storing in 8-bit instead of 4-byte. To implement this function, the key challenge is to extract four 8-bit numbers from every A[i][k]. For example, when A[0][0] = 0x01020304, it does not indicate a 4-byte unsigned integer 16909060. Instead, it represents four 8-bit numbers [0x01, 0x02, 0x03, 0x04] = [1, 2, 3, 4]. And if you have B[0:4][0] = [a, b, c, d], then C[0][0] = (1*a + 2*b + 3*c + 4*d).

To extract the four 8-bit numbers from A[i][k], you may consider using Boolean masking (*https://www.geeksforgeeks.org/c/extract-bits-in-c/*). As a hint, if you want to extract the last 8-bit number (0x04) from A[0][0] = 0x01020304, you can use **A[0][0] & 0xFF**.

For this question, we only grade the function correctness and **do not** require you to profile the `matmul_quant` function.

3.3 Attention

You are also required to complete the attention function:

```
float **scaled_dot_product_attention(float **Q, float **K, float **V,
int seqLength, int depth)
```

4. Report

The report will be **3-pages max** but can be shorter if you do not need all three pages. Report requirements/prompts:

- Include a horizontal segmented bar chart for the Top-down analysis of some (~2-3) of the different implementations (i.e. tile sizes, etc.) and different test inputs of `matmul_blocking` you tried. In the same bar chart, for the same set of inputs, also include the Top-down analysis data for `matmul` from Lab 1 to serve as a baseline.
- For the same implementations (incl. `matmul` from Lab 1) and inputs, collect their performance data (runtime) and include it in a table in the report.
- Explain in detail your profiling process.
- With reference to computer architecture concepts and the Top-down analysis data, explain the performance differences between the different implementations and inputs.
- Discuss performance differences between `matmul_blocking` and `matmul`.
- Discuss any issues you had during the lab and your debugging process.

5. Deliverables

Please submit your assignment on Canvas by **Oct. 23, 11:59 pm**. You are expected to submit your code and scripts in a single file named **lab2.zip** and your report in a file named **lab2.pdf**.

The lab will be marked out of a **total of 100 marks.**

- **20 marks** for `matmul_blocking()`
    - 10 marks for functional correctness (computes correct outputs for given set of inputs)
    - 10 marks for correct implementation of tiling/blocking
- **20 marks** for `matmul_quant()`
    - 20 marks for functional correctness (computes correct outputs for given set of inputs)
- **20 marks** for `scaled_dot_product_attention()`
    - 20 marks for functional correctness (computes correct outputs for given set of inputs)
- **40 marks** for report
    - 20 marks for Top-down analysis data
    - 10 marks for performance (runtime) data
    - 10 marks for discussion (see prompts in 4. Report)